EXPERIMENT NO: 11

Experiment to understand pandas library use cases in Data science.

PROGRAM:

```
import numpy as np
import pandas as pd
list=[[1,'Smith',50000],[2,'Jones',60000]]
df=pd.DataFrame(list)
df
```

OUTPUT:

	0	1	2
0	1	Smith	50000
1	2	Jones	60000

PROGRAM:

```
df.columns=['Empd','Name','Salary']
df
```

OUTPUT:

	Empd	Name	Salary
0	1	Smith	50000
1	2	Jones	60000

PROGRAM:

```
df.info()
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 180.0+ bytes
```

PROGRAM:

```
df=pd.read_csv("/content/50_Startups.csv")
df.info()
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype  
---  ------          --------------  -----  
 0   R&D Spend       50 non-null     float64
 1   Administration  50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State           50 non-null     object 
 4   Profit          50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB
```

PROGRAM:

```
df.head()
```

OUTPUT:

	R&D Spend	Administration	Marketing Spend	State	Profit
0	74308.32	180893.92	29300.28	New York	68422.31
1	157853.57	154642.93	304566.74	New York	166448.03
2	126139.12	176832.36	158031.97	Florida	123038.04
3	106805.48	170801.69	246399.66	California	123228.12
4	42622.70	130716.50	427942.75	California	88386.53

PROGRAM:

```
df.tail()
```

OUTPUT:

	R&D Spend	Administration	Marketing Spend	State	Profit
45	116065.73	116662.41	31783.56	California	94265.10
46	65198.11	120568.93	292351.77	California	93281.48
47	95409.86	107718.04	243718.96	Florida	111255.33
48	99272.99	53431.58	38422.83	Florida	80461.01
49	46803.90	64565.34	141784.14	California	58761.20

PROGRAM:

```
import numpy as np
import pandas as pd
df=pd.read_csv("/content/employee.csv")
df.head()
```

OUTPUT:

	emp_id	name	salary
0	1	SREE VARSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH H G	8000

PROGRAM:

```
df.tail()
```

OUTPUT:

	emp_id	name	salary
0	1	SREE VARSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH H G	8000

PROGRAM:

```
df.info()
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp_id  5 non-null      int64
 1   name    5 non-null      object
 2   salary  5 non-null      int64
dtypes: int64(2), object(1)
memory usage: 252.0+ bytes
```

PROGRAM:

df.salary

OUTPUT:

	salary
0	5000
1	6000
2	7000
3	5000
4	8000

dtype: int64

PROGRAM:

type(df.salary)

OUTPUT:

pandas.core.series.Series

def __init__(data=None, index=None, dtype: Dtype | None=None, name=None, copy: bool | None=None, fastpath: bool | lib.NoDefault=lib.no_default) -> None

One-dimensional ndarray with axis labels (including time series).

Labels need not be unique but must be a hashable type. The object supports both integer- and label-based indexing and provides a host of methods for performing operations involving the index. Statistical methods from ndarray have been overridden to automatically exclude

PROGRAM:

df.salary.mean()

OUTPUT:

```
np.float64(6200.0)
```

PROGRAM:
df.salary.median()

OUTPUT:

```
6000.0
```

PROGRAM:
df.salary.mode()

OUTPUT:

	salary
0	5000

dtype: int64

PROGRAM:
df.salary.var()

OUTPUT:

```
1700000.0
```

PROGRAM:
df.salary.std()

OUTPUT:

```
1303.8404810405298
```

PROGRAM:
df.describe()

OUTPUT:

	emp_id	salary
count	5.000000	5.000000
mean	3.000000	6200.000000
std	1.581139	1303.840481
min	1.000000	5000.000000
25%	2.000000	5000.000000
50%	3.000000	6000.000000
75%	4.000000	7000.000000
max	5.000000	8000.000000

PROGRAM:

```
df.describe(include='all')
```

OUTPUT:

	emp_id	name	salary
count	5.000000	5	5.000000
unique	NaN	5	NaN
top	NaN	SREE VARSINI K S	NaN
freq	NaN	1	NaN
mean	3.000000	NaN	6200.000000
std	1.581139	NaN	1303.840481
min	1.000000	NaN	5000.000000
25%	2.000000	NaN	5000.000000
50%	3.000000	NaN	6000.000000
75%	4.000000	NaN	7000.000000
max	5.000000	NaN	8000.000000

PROGRAM:

```
empCol=df.columns
empCol
```

OUTPUT:

```
Index(['emp_id', 'name', 'salary'], dtype='object')
```

PROGRAM:

```
emparray=df.values
emparray
```

OUTPUT:

```
array([[1, 'SREE VARSINI K S', 5000],
       [2, 'SREEMATHI B', 6000],
       [3, 'SREYA G', 7000],
       [4, 'SREYASKARI MULLAPUDI', 5000],
       [5, 'SRI AKASH H G', 8000]], dtype=object)
```

PROGRAM:

```
employee_DF=pd.DataFrame(emparray,columns=empCol)
employee_DF
```

OUTPUT:

	emp_id	name	salary
0	1	SREE VARSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH H G	8000